LAB 11: Software Metrics and Complexity Analysis

Smart Waste Collection Management System

KIET Deemed To Be University | Computer Science Engineering | CSE-E

Prepared by: Tanishka Singh - Leader (Product Manager)

1. Introduction and scope

This report measures the size, quality and expected effort of the Smart Waste Collection Management System. Besides the UML design of Labs 2 to 9, the project now has a working prototype (the `smart_waste` package: backend services, REST API, sensor simulator, dashboard and tests). The metrics are therefore measured on real code and not only estimated. All code metrics were produced by the script `tools/measure_metrics.py` that is included with the project, so every number can be reproduced.

Metric group	Measured on	Metrics
Size	UML models, SRS, source code	Classes, methods, links, requirements, lines of code, tests
Object-oriented design	Class diagram and real classes	WMC, DIT, NOC, CBO, LCOM (Chidamber and Kemerer suite)
Architecture	Component diagram and module imports	Afferent / efferent coupling and instability
Complexity	Behaviour diagrams and real code	McCabe cyclomatic complexity, Halstead measures, maintainability index
Testing	Test suite	Number of tests, pass rate, line coverage, test-to-code ratio
Estimation	SRS requirements and measured size	Function points, basic COCOMO effort, time and team size

2. Size metrics

2.1 Design size

Item	Count	Source
Classes	11	Lab 3 class diagram
Attributes / methods	40 / 27	Lab 3 class diagram
Links between classes	13 (3 generalisations, 10 associations, aggregation and dependency)	Lab 3 class diagram
Use cases / actors	14 / 7	Lab 2 use case diagram
UML diagrams produced	18 (1 use case, 1 class, 2 activity, 5 sequence, 3 collaboration, 4 state, 2 component)	Labs 2 to 9
Functional requirements	15 (REQ-BM 3, REQ-AL 3, REQ-RO 3, REQ-DB 3, REQ-CF 3)	SRS section 4
Non-functional and business rule statements	15 (performance 3, safety 2, security 3, quality 4, business rules 3)	SRS section 5

2.2 Implementation size

File	Lines of code	Functions	Avg V(G)	Max V(G)	Line coverage
models.py	75	2	1.50	2	100%
core.py	63	10	2.70	5	100%
db.py	22	1	1.00	1	100%
services.py	282	41	2.20	7	94%
server.py	162	31	1.77	7	81%
simulator.py	47	4	2.75	6	84%
Package total	651	89	2.10	7	91.3%

In addition there are 172 lines of HTML, CSS and JavaScript (dashboard) and 233 lines of test code in 37 tests. The test code is 36% of the size of the package source.

3. Object-oriented design metrics (Chidamber and Kemerer)

WMC is the sum of the cyclomatic complexity of the methods of a class (in the design table every method counts 1, because bodies did not exist yet). DIT is the depth in the inheritance tree and NOC the number of direct subclasses. CBO is the number of other project classes a class is coupled to (inheritance not counted). LCOM is the number of method pairs that share no attribute minus the pairs that share one (minimum 0); a high value means the class could be split.

3.1 From the design (Lab 3 class diagram)

Class	WMC	DIT	NOC	CBO
User (abstract)	2	0	3	1
Administrator	4	1	0	2
Driver	2	1	0	1
Citizen	2	1	0	1
Bin	3	0	0	4
SensorNode	4	0	0	2
SensorReading	1	0	0	1
Alert	3	0	0	2
Route	3	0	0	3
Complaint	2	0	0	2
Report	1	0	0	1
Total / average	27 / 2.45	max 1 / 0.27	3 / 0.27	20 / 1.82

3.2 From the implementation (classes with behaviour)

Class	Methods	WMC	CBO	LCOM	DIT
Bin	2	3	0	1	0
Settings	3	4	0	0	0
LogNotifier	2	2	0	0	0
AuthService	5	12	4	2	0
BinRepository	5	6	2	0	0
AlertService	5	10	1	0	0
IngestionService	3	12	0	0	0
ComplaintService	5	10	2	0	0
RouteService	8	22	2	0	0
AnalyticsService	2	5	0	0	0
App	2	6	9	0	0
Handler	10	29	1	39	0
Average (12 classes)	4.3	10.1	1.8	3.5	-

- **The domain classes** of the model (User, Administrator, Driver, Citizen, SensorNode, SensorReading, Alert, Route, Complaint, Report) are implemented as data classes without logic; only Bin has behaviour. The logic lives in the service classes, as in the Lab 9 component diagram. Inheritance depth is 1 for the user roles and 0 elsewhere.
- **WMC:** most service classes are between 2 and 12. RouteService and Handler are above the usual guideline of 20 (RouteService 22, Handler 29). RouteService does route planning and collection handling together and the HTTP Handler mixes request parsing, routing and responses; both are candidates for splitting.

- **CBO:** low everywhere except App (9), which is only the place where all services are created and wired together (composition root).
- **LCOM:** zero for all service classes. Handler has 39 because its helper methods do not share state; moving them to functions would lower it.

4. Architecture metrics: coupling and instability

Ca is the number of modules that depend on a module, Ce the number it depends on, and instability I = Ce / (Ca + Ce). A value near 0 means a stable module that many others rely on; near 1 a module that is free to change.

4.1 Backend modules in the design (Lab 9 component diagram)

Module	Ca	Ce	Instability I
REST API Layer	0	6	1.00
Auth Service	1	1	0.50
Data Ingestion Service	1	1	0.50
Alert Service	1	2	0.67
Route Optimization	1	2	0.67
Complaint Service	1	1	0.50
Analytics Service	1	1	0.50
Notification Adapter	1	0	0.00
Map Adapter	1	0	0.00
Data Access Layer	6	1	0.14

4.2 Python modules in the implementation (from imports)

Module	Ca	Ce	Instability I	Role
core.py	3	0	0.00	Pure logic, imports nothing
models.py	1	0	0.00	Domain data classes
db.py	1	0	0.00	Storage setup
services.py	1	2	0.67	Business services
server.py	0	3	1.00	HTTP API and dashboard
simulator.py	0	1	1.00	Sensor simulator

In both views dependencies point from unstable modules (REST API Layer / server, I = 1.00) towards stable ones (Data Access Layer, adapters / core, models, db, I = 0.00 to 0.14). The code therefore follows the design and the stable dependencies principle: the algorithms in core.py can change without touching the server, and no module depends on the server or the simulator.

5. Complexity metrics

5.1 Cyclomatic complexity from the behaviour diagrams

For an activity diagram V(G) = number of decision nodes + 1. For a state machine V(G) = transitions - states + 2, where the initial node, the final node and choice nodes are counted as states.

Diagram	Decisions / transitions and states	V(G)	Meaning
Activity 1: monitoring to collection	4 decisions (reading valid, fill at or above threshold, adjust route, more bins)	5	Simple; 5 independent test paths
Activity 2: complaint handling	0 decisions (fork and join only)	1	Linear, parallel branches
State machine: Bin	10 transitions, 7 states	5	Simple
State machine: Sensor Node	8 transitions, 6 states	4	Simple
State machine: Route	9 transitions, 8 states	3	Simple
State machine: Complaint	5 transitions, 6 states	1	Linear

5.2 Measured complexity of the code

V(G) counts each if, loop, comprehension, comprehension condition, except handler and boolean operator plus 1, for each of the 89 functions and methods (nested functions are counted on their own). The maintainability index is MI = max(0, (171 - 5.2 ln V - 0.23 V(G) - 16.2 ln LOC) x 100 / 171), where V is the Halstead volume of the function.

V(G) band	Functions	Share	Risk
1 to 5	85	96%	Simple, easy to test
6 to 10	4	4%	Moderate
above 10	0	0%	Complex, should be split

The eight most complex functions:

Function	File	LOC	V(G)	MI
Handler._dispatch	server.py	20	7	49.2
IngestionService.record	services.py	16	7	51.7
RouteService.edit	services.py	13	7	54.6
run	simulator.py	19	6	49.5
edit_route	core.py	8	5	62.8
AuthService.require	services.py	8	5	61.7
AnalyticsService.report	services.py	13	4	54.4
App.seed_demo	services.py	12	4	54.4

Finding and fix. The first measurement showed one function above the limit: Handler._dispatch in server.py had V(G) = 16 and MI = 44.5, because it parsed the request, found the route, checked the login and built the error responses in one place. It was split into small helper methods (_read_json, _parse_query, _find_route, _call). Its V(G) is now 7, the highest value in the package is 7 and all 37 tests still pass. The average V(G) is 2.10 and the average maintainability index is 69.1 (a value of 20 or more is considered good). The lowest MI values belong to the two factory functions make_routes and make_handler (both about 35); they are long because they only list the routes, not because they are complicated.

5.3 Halstead measures for the whole package

Measure	Value	Measure	Value
Distinct operators n1	48	Program volume V = N log2 n	54824
Distinct operands n2	651	Difficulty D = (n1/2)(N2/n2)	89.7
Total operators N1	3368	Effort E = D x V	4.92e+06
Total operands N2	2434	Time T = E / 18 seconds	76 hours
Vocabulary n / length N	699 / 5802	Delivered bugs B = V / 3000	18.3

Halstead's formulas are rough: the large vocabulary is mostly names, strings and SQL text, so the effort and bug figures should be read as an upper estimate. The test suite found no failing behaviour, but the estimate of about 18 latent defects is a reminder to keep testing as the system grows.

6. Testing metrics

Item	Result
Automated tests	37 (core logic, services and REST API, including the simulator)
Result	37 passed, 0 failed
Line coverage of the package	91.3% (554 of 607 executable lines)
Test code to source code	233 : 651 lines (ratio 0.36)
Tests per 100 lines of source	5.7
Known open defects	0

File	Covered / executable lines	Coverage
models.py	75 / 75	100%
core.py	57 / 57	100%
db.py	8 / 8	100%
services.py	249 / 265	94%
server.py	128 / 158	81%
simulator.py	37 / 44	84%

Models, core logic and database setup are fully covered. The uncovered lines are error branches in services.py and server.py (for example a malformed request body) and parts of the simulator command line. Coverage was measured with a small line tracer based on the standard library; the numbers are statement coverage, not branch coverage.

7. Function point analysis

Function points are counted from the SRS. Each item is rated Low, Average or High and given the standard IFPUG weight.

Component	Rating	Weight
EI: Submit sensor reading	Average	4
EI: Configure alert threshold	Low	3
EI: Request route (admin / daily trigger)	Average	4
EI: Add / remove bin on route	Average	4
EI: Submit complaint with photo	Average	4
EI: Mark bin as collected	Low	3
EI: Administrator login	Low	3
EO: Threshold alert notification	Average	5
EO: Optimized route output (calculated)	High	7
EO: Collections / average fill-level report	Average	5
EQ: View live bin map and status	Average	4
EQ: View bin fill-level history	Average	4
EQ: View assigned route (driver)	Low	3
EQ: View complaint status	Low	3
ILF: Bins and sensor readings	Average	10
ILF: Routes	Average	10
ILF: Complaints	Average	10
ILF: Users / accounts	Low	7
ILF: Alert log	Low	7
EIF: Map and distance data (OSM / Google Maps)	Average	7
EIF: Bin location and route data from municipality	Low	5
Unadjusted function points (UFP)		112

Subtotals: EI 25, EO 17, EQ 14, ILF 44, EIF 12; total UFP = 112.

General system characteristic	Rating (0-5)	General system characteristic	Rating (0-5)
Data communications	5	Online update	3
Distributed data processing	4	Complex processing	3
Performance	3	Reusability	2
Heavily used configuration	2	Installation ease	2

General system characteristic	Rating (0-5)	General system characteristic	Rating (0-5)
Transaction rate	3	Operational ease	3
Online data entry	4	Multiple sites	3
End-user efficiency	3	Facilitate change	4

Total degree of influence = 44, VAF = 0.65 + 0.01 x 44 = 1.09, so **FP = 112 x 1.09 = 122 function points**. The ratings are my own judgement for this project.

How much of this is built? The prototype implements all 7 inputs, all 3 outputs, 3 of the 4 inquiries (bin history is missing) and all 5 internal files; the two external files are replaced by the haversine formula and sample data. That is 96 of 112 unadjusted function points, or about **86%** of the functionality.

8. Effort estimation (basic COCOMO)

Before coding, the size was estimated as FP x 50 lines per function point (a typical value for Java-like languages) = 6.1 KLOC. **After coding**, the measured size is 651 lines (Python) + 172 lines (HTML / JavaScript) = 823 lines for about 105 adjusted function points, i.e. only **7.9 lines per function point**. Python with the standard library is much more compact than the assumed 50, so the estimate is repeated with the measured productivity.

Estimate	Size	Mode	Effort (person-months)	Schedule (months)	Team
Before coding (50 LOC / FP)	6.1 KLOC	Semi-detached	22.8	7.5	3.0
Before coding (50 LOC / FP)	6.1 KLOC	Organic	16.0	7.2	2.2
After coding (measured 7.9 LOC / FP)	0.96 KLOC	Organic	2.3	3.4	0.7

The re-estimate (about 2.3 person-months over 3.4 months) fits the way the prototype was built: a small team working for about one semester. The earlier, larger figures still apply to a production version with Java or JavaScript, two native mobile apps, ESP32 firmware, real FCM and map integration, and security hardening.

9. Summary of results

Metric	Result	Accepted range (guideline)	Verdict
Design WMC / DIT (maximum)	4 / 1	20 / 5 or less	Good
Design CBO (average / maximum)	1.82 / 4	5 or less	Good; watch Bin
Code WMC (maximum)	29 (Handler)	20 or less	Two classes above; split
Code LCOM (services)	0	Low	Good
Module instability	Flows from I = 1.00 to I = 0.00	Depend towards stability	Good
Cyclomatic complexity (max, code)	7 (was 16 before refactoring)	10 or less	Good
Average cyclomatic complexity	2.10	Low	Good
Maintainability index (average)	69.1	20 or more	Good
Tests / pass rate	37 / 100%	All pass	Good
Line coverage	91.3%	80% or more	Good
Function points	122 FP (86% built)	-	Medium-small system
Effort (re-estimate / full product)	2.3 / 22.8 person-months	-	Feasible for a small team

10. Conclusion and recommendations

Measured on real code, the project is small, simple and well tested: 651 lines of Python, an average V(G) of 2.10, no function above V(G) 7 after one refactoring, an average maintainability index of 69.1, 37 passing tests and 91% line coverage. The implementation follows the design: the same modules, the same stable-to-unstable dependency direction, and about 86% of the function points. Recommendations:

- Split RouteService (route planning vs. collection handling) and move the helper methods out of the HTTP Handler to bring WMC below 20.
- Add tests for the uncovered error branches and track branch coverage as well, using a coverage tool once packages can be installed.
- Add the missing functions: bin history inquiry, a real mapping API adapter, a real FCM adapter, authentication for the sensor endpoint and driver accounts.
- Re-run `python tools/measure_metrics.py` after each change and keep every function at V(G) of 10 or less.

Appendix: Formulas used

- Cyclomatic complexity: V(G) = decisions + 1 (code, activity diagrams); V(G) = transitions - states + 2 (state machines).
- Halstead: N = N1 + N2; n = n1 + n2; V = N log2 n; D = (n1/2)(N2/n2); E = D x V; T = E / 18; B = V / 3000.
- Maintainability index: MI = max(0, (171 - 5.2 ln V - 0.23 V(G) - 16.2 ln LOC) x 100 / 171).
- Function points: FP = UFP x (0.65 + 0.01 x total degree of influence).
- Basic COCOMO: Effort = a x KLOC^b person-months; Time = c x Effort^d months; Team = Effort / Time (organic a = 2.4, b = 1.05, c = 2.5, d = 0.38; semi-detached a = 3.0, b = 1.12, c = 2.5, d = 0.35).
- Instability: I = Ce / (Ca + Ce). LCOM = max(P - Q, 0), P = method pairs sharing no attribute, Q = pairs sharing one.